학습 목표

하네스를 '어디에 개입해 무엇을 통제하는가'의 지도로 읽는다

1. **하네스 구성 요소 8가지 이상을 나열하고 개입 지점을 표시한다**
   각 요소가 행동 전, 행동 중, 행동 후 중 어디에 개입하는지 구분

2. **통제 장치를 가이드/센서 × 계산적/추론적 2×2에 배치한다**
   AGENTS.md, ArchUnit, LLM 코드 리뷰, 타입체커, 스킬 등

3. **가이드나 센서 한쪽만 있을 때의 실패 양상을 예로 설명한다**
   규칙만 있는 팀과 검사만 있는 팀의 차이

4. **유지보수성, 아키텍처 적합성, 행위 하네스를 구분한다**
   행위 하네스가 아직 미해결 과제인 이유까지 설명

> ***이 장의 2×2 프레임이 3장부터 설계 작업의 기본 틀이 된다***

무단전재 및 배포 금지

도입 사례: 한쪽 날개 하네스

규칙은 있었는데 왜 계속 틀렸을까(이해를 돕는 가상 사례)

A팀: 규칙만

- AGENTS.md에 300줄 넘는 규칙
- PR에 규칙 위반 코드가 꾸준히 섞임
- 위반은 사람 리뷰에서만 발견
- 지켜졌는지 확인하는 장치가 없음

B팀: 검사만

- 린트, 타입체크, 테스트가 촘촘함
- 에이전트에게 주는 사전 안내는 거의 없음
- 같은 규칙에 반복해 걸리고 재시도
- 다음 세션에서도 같은 실수 반복

진단

- 두 팀 모두 한쪽 날개만 단 하네스
- 가이드(피드포워드)와 센서(피드백)
- 어디에 개입해 무엇을 통제하는가

> ***구성 요소 목록이 아니라 개입과 통제의 지도로 본다***

출처: Birgitta Böckeler, "Harness engineering for coding agent users", martinfowler.com, 2026-04-02

무단전재 및 배포 금지

에이전트 루프와 개입 지점

모델 호출, 도구 실행, 관찰을 반복하며 하네스가 곳곳에 개입한다

1 **컨텍스트 조립 (행동 전)**
시스템 프롬프트, AGENTS.md, 스킬 메타데이터, 도구 정의를 모델 입력에 넣습니다.

2 **모델 호출**
쌓인 컨텍스트로 다음 행동을 고릅니다. 모델이 직접 하는 일은 이 단계뿐입니다.

3 **도구 실행 (행동 중)**
사전 훅과 권한 검사를 거쳐 샌드박스에서 실행하고, 사후 훅으로 포맷과 린트를 돌립니다.

4 **관찰 (행동 중)**
실행 결과를 컨텍스트에 추가합니다. 컴팩션과 큰 출력 오프로딩이 여기서 동작합니다.

5 **정지 훅 (행동 후)**
완료 선언 시 타입체크, 테스트, 체크리스트로 검사합니다. 실패하면 재작업, 통과하면 커밋과 PR.

> ***Schmid: 모델은 CPU, 컨텍스트는 RAM, 하네스는 OS***

출처: Philipp Schmid, "The importance of Agent Harness in 2026", 2026-01-05

무단전재 및 배포 금지

구성 요소 지도 (1/2)

Trivedy와 Osmani의 목록을 합친 10가지 공통 어휘

하네스 구성 요소 1~5

구성 요소	하는 일	개입 지점	자세히
시스템 프롬프트	역할, 작업 방식, 검증 습관 지정	행동 전	3장
지침 파일	세션마다 주입되는 레포 수준 안내	행동 전	3장
스킬	요약만 보이다 필요할 때 본문을 읽음	행동 전(필요 시 중)	3장
도구와 MCP	에이전트가 세상에 손을 뻗는 수단	행동 중	4장
파일시스템과 git	결과와 진행 상태를 세션 밖에 저장	행동 중, 후	7장

1. 지침 파일 예: AGENTS.md, CLAUDE.md, 디렉터리별 규칙 파일
2. MCP(Model Context Protocol)는 외부 도구를 에이전트에 연결하는 표준 규약
3. 도구는 많은 것 중 작업에 맞는 일부만 붙인다 (예: Stripe의 Toolshed)
4. 파일시스템 예: 진행 파일, JSON 기능 목록, 설명이 담긴 커밋

출처: Vivek Trivedy, "The Anatomy of an Agent Harness", LangChain Blog, 2026-03-10; Addy Osmani, "Agent Harness Engineering", 2026-04-19

무단전재 및 배포 금지

구성 요소 지도 (2/2)

하네스는 대부분 레포 안의 코드와 설정 파일로 남는다

하네스 구성 요소 6~10

구성 요소	하는 일	개입 지점	자세히
샌드박스	격리된 작업장(devbox, 컨테이너)	행동 중	4장
훅과 미들웨어	특정 시점에 끼우는 결정적 코드	행동 중, 후	5장
서브에이전트	별도 컨텍스트로 처리 후 요약 반환	행동 중	6장
컨텍스트 정책	컨텍스트 창에 넣고 뺄 것 결정	행동 중	6장
복구 경로	실패했을 때 돌아갈 길	행동 후	5장, 7장

1. 훅 예: 위험 명령 거부, 정지 시 타입체크, LoopDetection
2. 복구 경로 예: git 이력으로 되돌리기, 재시도 상한, 사람에게 넘기기
3. 훅과 권한은 .claude/settings.json, 스킬은 .claude/skills/ 아래에 둔다
4. 레포 안에 남으므로 버전 관리, 리뷰, 공유가 가능하다(10장 템플릿)

> ***하네스는 버전 관리되는 팀 자산이다***

출처: Addy Osmani, "Agent Harness Engineering", 2026-04-19

무단전재 및 배포 금지

제품 하네스와 사용자 하네스

제품이 기본 제공하는 층 위에 팀이 자기 층을 덧붙인다

사용자 하네스

- 팀이 덧붙이는 바깥 층
- AGENTS.md와 docs, 스킬
- 훅 스크립트, 커스텀 린터
- CI 파이프라인, 리뷰 정책

제품 하네스

- Claude Code, Codex, 에이전트 SDK
- 에이전트 루프, 기본 도구
- 권한 모델, 컴팩션, 훅 지점

모델

- 사후 학습된 가중치
- 자기 제품 하네스를 끼운 채 학습되기도 함

실무 함의

- 바꿀 수 없는 제품 동작은 두고 빈 곳을 메운다
- 도구 구성을 크게 바꾸면 성능 저하 가능
- 하네스 과적합은 9장에서 측정

> ***대부분의 팀이 설계할 수 있는 곳은 사용자 하네스다***

출처: Birgitta Böckeler, "Harness engineering for coding agent users", 2026-04-02; Vivek Trivedy, "The Anatomy of an Agent Harness", 2026-03-10

무단전재 및 배포 금지

가이드와 센서

첫 번째 축: 각 부품이 어떤 방식으로 통제하는가

가이드

- 피드포워드: 결과 보기 전에 입력 조정
- 비유: 출발 전 내비게이션 경로 안내
- AGENTS.md, 스킬, 코딩 규약
- 코드모드(예: OpenRewrite)

센서

- 피드백: 결과를 관찰한 뒤 교정
- 비유: 차선 이탈 경고음
- 테스트, 린터, 타입체커
- ArchUnit, 리뷰 에이전트

구분 기준

- 관찰한 결과에 반응하는가
- 결과에는 도구 호출과 루프 상태도 포함
- 그래서 센서는 행동 중에도 동작

> ***가이드와 센서를 가르는 기준은 시점이 아니다***

출처: Birgitta Böckeler, "Harness engineering for coding agent users", martinfowler.com, 2026-04-02

무단전재 및 배포 금지

짝이 없을 때 생기는 실패

가이드와 센서는 서로의 약점을 메운다

가이드와 센서의 짝 상태별 양상

상태	증상	흔한 잘못된 대응
가이드만 있음	위반이 늘고 사람이 리뷰에서 발견	지침을 늘리거나 CRITICAL 강조
센서만 있음	같은 검사에 반복 실패, 재시도 비용 누적	재시도 횟수를 늘림
둘 다 있고 짝지어짐	센서 실패 메시지로 스스로 수정	해당 없음(바람직한 상태)

1. 가이드만: 규칙이 지켜졌는지 아무도 확인하지 않는다
2. 센서만: 피할 것을 미리 몰라 다음 세션에도 같은 실수를 한다
3. 짝지어지면 가이드로 대부분 맞게 만들고, 놓친 것은 센서가 잡는다
4. 가이드는 있다는 것만으로 효과가 보장되지 않는다(Gloaguen 외, 3장)

> ***강조 문구는 규칙이 지켜졌는지 확인해 주지 않는다***

출처: Gloaguen 외, "Evaluating AGENTS.md: Are Repository-Level Context Files Helpful for Coding Agents?", arXiv:2602.11988

무단전재 및 배포 금지

짝 맞추기: 레이어 규칙 예시

AGENTS.md의 레이어 규칙을 ArchUnit으로 같은 범위만큼 검사한다

java

```
@AnalyzeClasses(packages = "com.example.shop")
class LayerRulesTest {
  @ArchTest
  static final ArchRule
    domain_must_not_depend_on_other_layers =
    noClasses().that().resideInAPackage("..domain..")
      .should().dependOnClassesThat()
      .resideInAnyPackage("..api..", "..application..",
                          "..infrastructure..")
      .because("domain은 다른 레이어에 의존하지 않습니다. "
        + "domain/port에 인터페이스를 선언하고 "
        + "infrastructure에서 구현하세요. "
        + "예시: docs/architecture/layers.md");
}
```

1. 가이드: AGENTS.md의 의존 방향 규칙(api → application → domain ← infrastructure)과 layers.md 링크
2. because에 고치는 방법과 문서 위치를 담아 센서가 다시 가이드를 가리키게 한다
3. 센서의 검사 범위는 가이드와 같아야 한다(application 규칙도 같은 방식으로 검사)
4. 같은 위반이 반복되면 가이드를 보강한다(Hashimoto: 실수 하나에 영구 해결책 하나)

> ***센서 실패 메시지가 가이드로 이어지면 순환이 완성된다***

출처: Mitchell Hashimoto, "My AI Adoption Journey", 2026-02-05

무단전재 및 배포 금지

계산적 통제와 추론적 통제

두 번째 축: 통제가 무엇으로 실행되는가

계산적 통제와 추론적 통제 비교

비교 항목	계산적 통제	추론적 통제
실행 방식	CPU에서 도는 결정적 검사	LLM이 판단
비용	거의 들지 않음	호출마다 토큰 비용
속도	보통 수 초 이내	대개 훨씬 느림
결정성	같은 입력이면 같은 결과	실행마다 다를 수 있음
판단 범위	규칙으로 표현 가능한 것만	의도, 가독성, 설계 적절성
실행 빈도	모든 변경, 커밋 전마다	중요한 지점에서 선택적
예	테스트, 린트, 타입체크	AI 코드 리뷰, LLM-as-judge

1. **계산적 센서를 먼저 깔고**, 규칙으로 못 잡는 것만 추론적 센서에 맡긴다

2. 가이드에도 같은 축: AGENTS.md와 스킬은 추론적, 스캐폴딩과 코드모드는 계산적

3. 분류 기준: 효과를 내는 과정에 LLM의 해석이나 판단이 끼어드는가

> ***두 방식은 대체가 아니라 보완 관계다***

출처: Birgitta Böckeler, "Harness engineering for coding agent users", 2026-04-02; Alistair Gray, "Minions", Stripe Dot Dev Blog, 2026-02-09

무단전재 및 배포 금지

2×2 매트릭스로 배치하기

두 축을 겹친 네 칸이 3장부터 이어지는 설계의 기본 틀이다

가이드/센서 × 계산적/추론적

구분	계산적	추론적
가이드	스캐폴딩 스크립트, 서비스 템플릿	AGENTS.md, CLAUDE.md, docs/
가이드	코드모드(OpenRewrite), 타입 정보	스킬, 시스템 프롬프트 작업 지침
가이드	-	PreCompletionChecklist
센서	타입체커, 린터, 단위와 통합 테스트	LLM 코드 리뷰, 리뷰 서브에이전트
센서	ArchUnit, 커스텀 린터	LLM-as-judge 평가기
센서	PreToolUse 위험 명령 거부 훅	Playwright 평가 에이전트
센서	LoopDetection(반복 편집 감지)	-

1. 훅과 미들웨어는 칸이 아니라 **배선**: 무엇을 실행하느냐로 칸이 정해진다
2. PreCompletionChecklist는 완료 직전에 끼지만 점검 항목을 상기시키는 추론적 가이드
3. 서브에이전트는 역할에 따라 다르다: 탐색은 컨텍스트 관리, 리뷰는 추론적 센서
4. 중요한 규칙일수록 여러 칸에서 겹쳐 지킨다

> ***가이드/센서 축과 개입 시점은 서로 독립이다***

출처: Birgitta Böckeler, "Harness engineering for coding agent users", 2026-04-02; Vivek Trivedy, "Improving Deep Agents with harness engineering", 2026-02-17

무단전재 및 배포 금지

규제 대상의 세 범주

2×2는 통제의 방식을, 세 범주는 통제의 대상을 나눈다

하네스가 지키려는 대상

범주	지키려는 것	대표 센서	성숙도
유지보수성	스타일, 복잡도, 중복, 타입 안전성	린터, 포매터, 타입체커	현재 가장 쉬움
아키텍처 적합성	레이어 의존 방향, 모듈 경계	ArchUnit, 커스텀 린터	규칙을 팀이 직접 정의
행위	요구사항대로 올바르게 동작	기능 테스트, E2E, 평가 에이전트	미해결

1. 대표 가이드는 차례로 코딩 규약, 아키텍처 문서와 서비스 템플릿, 사양서와 인수 조건
2. 아키텍처 적합성에는 관측성, 성능 같은 구조적 요구도 포함된다
3. OpenAI: 커스텀 린터로 단방향 레이어 의존 강제, 유지보수성과 아키텍처가 두텁다
4. Böckeler: 그 글은 기능과 행위의 검증을 다루지 않았다고 지적

> ***하네스가 가장 약한 곳은 행위의 올바름이다***

출처: Birgitta Böckeler, "Harness Engineering - first thoughts", martinfowler.com, 2026-02-17; OpenAI, "Harness Engineering: Leveraging Codex in an Agent-First World"

무단전재 및 배포 금지

행위 하네스는 왜 어려운가

구조적 이유 네 가지와 현재의 부분적 보강

정답이 코드 밖
- 판정하려면 요구사항을 알아야 함
- 요구사항은 모호하거나 문서 없음
- 예: 쿠폰은 주문당 하나만 적용

센서 순환 문제
- 에이전트가 코드와 테스트를 함께 작성
- 잘못 이해하면 둘 다 틀린 채 통과
- AI 테스트를 믿을지 합의 없음

침묵의 해석
- 센서가 울리지 않는 이유가 모호
- 품질이 좋아서인가, 탐지력이 약한가
- 행위 영역에서 특히 심각

자기 평가 편향
- 자기 결과물을 후하게 평가하는 경향
- 같은 에이전트가 만들고 채점하면
- 행위 센서가 무뎌짐

부분적 보강
- 사람처럼 앱을 조작하는 E2E(7장)
- 생성기와 회의적 평가기 분리(8장)
- 완료의 정의를 코딩 전 계약으로

하네스의 목표
- 사람을 빼는 것이 아님
- 사람의 주의를 중요한 곳으로 보냄
- 사람은 행위의 올바름에 집중

> ***행위 센서 설계는 결국 에이전트 평가(evals) 설계다***

출처: Anthropic Engineering, "Harness design for long-running application development", 2026-03-24; "Demystifying evals for AI agents", 2026-01-09

무단전재 및 배포 금지

핵심 정리

1. **모델이 직접 하는 일은 모델 호출뿐이고, 나머지 루프는 모두 하네스가 담당한다**
   Schmid: 모델은 CPU, 컨텍스트는 RAM, 하네스는 OS

2. **하네스 구성 요소 10가지는 행동 전, 중, 후 중 개입하는 지점이 서로 다르다**

3. **제품 하네스와 사용자 하네스를 구분하고, 팀은 사용자 하네스를 설계한다**
   이 교재의 용어로 Harness-as-a-Service

4. **가이드는 행동에 앞서 방향을 제시하고, 센서는 결과를 관찰한 뒤 교정한다**
   이 축은 개입 시점과 독립적이다

5. **가이드와 센서는 짝짓고, 센서의 검사 범위는 가이드와 같아야 한다**
   실패 메시지가 가이드를 가리키면 순환이 완성된다

6. **계산적 센서를 먼저 깔고, 나머지만 추론적 통제에 맡긴다**

7. **훅과 미들웨어는 2×2의 칸이 아니라 배선이다**
   칸은 훅이 무엇을 실행하느냐로 정해진다

8. **규제 대상은 유지보수성, 아키텍처 적합성, 행위이며 행위 하네스는 미해결 과제다**

> ***방식(2×2), 시점(전, 중, 후), 대상(세 범주)의 세 축으로 진단한다***

무단전재 및 배포 금지

실습 안내

1장 인벤토리를 2×2에 옮겨 '현재 하네스 진단표'를 만든다

1

1. 인벤토리를 CSV로 정리

통제 장치마다 kind, mode, category, stage, pairs_with를 붙입니다. 훅은 훅이 실행하는 검사를 한 행으로 적습니다.

2

2. 진단 스크립트 실행

python -X utf8 harness_diag.py inventory.csv 로 2×2 배치, 짝 없는 항목, 범주별과 시점별 분포를 출력합니다.

3

3. 진단표 작성

빈 칸, 짝 없는 가이드와 센서, 가장 약한 범주, 얇은 개입 시점을 옮기고 '왜 비어 있는가'를 한 줄씩 씁니다.

4

4. 보강 우선순위 3가지

반복된 실수와 연결된 빈칸부터, 계산적 센서를 먼저, 기존 가이드에 센서 짝 맞추기를 우선합니다.

5

5. 확인

네 칸 합과 시점별 합이 행 수와 같은지, 우선순위마다 실제 실수 기록 근거가 있는지 점검합니다.

> ***결과물: 진단표 한 장과 보강 우선순위 3가지***

무단전재 및 배포 금지